CO3 - AT1

Case Study Analysis

Descriptive Statistics, Dispersion, Outliers & Distribution Shape

VISHAAL M S

Register No.: 192424420

Course: DSA0405 - Fundamentals of Data Sciences

Slot: D

Saveetha School of Engineering (SIMATS)

Methodology Note

Each case study below is solved using population-level descriptive statistics (mean, median, mode), measures of dispersion (range, variance, standard deviation, coefficient of variation) and quartile-based measures (Q1, Q3, IQR) where relevant. Outliers are flagged using the standard 1.5 x IQR rule: any value below Q1 - 1.5(IQR) or above Q3 + 1.5(IQR) is treated as an outlier. Variance and standard deviation are computed as population statistics (dividing by N) since each dataset represents the complete set of observations described in the scenario, not a sample drawn from a larger population.

$$\text{Mean } \mu = \Sigma x / N \qquad \text{Variance } \sigma^2 = \Sigma(x - \mu)^2 / N \qquad \text{Std. Dev. } \sigma = \sqrt{\sigma^2}$$

$$IQR = Q3 - Q1 \qquad \text{Outlier bounds} = [Q1 - 1.5 \times IQR,\ Q3 + 1.5 \times IQR] \qquad z = (x - \mu) / \sigma$$

Case Study 1 — Employee Salaries

Data (₹ lakhs, n = 9): 4, 5, 5, 6, 7, 8, 8, 9, 30

Mean	Median	Mode	Range	Variance	Std. Dev.
9.11	7	5	26	56.99	7.55

Outlier

The value 30 is a clear outlier — more than three times the next-highest salary and over 22 lakhs above the median. It lies far outside the IQR-based bound of 12.5 lakhs (Q1 = 5, Q3 = 8, IQR = 3).

Best Measure of Central Tendency

The median (₹7 lakhs) best represents the typical salary. The mean (₹9.11 lakhs) is pulled upward by the single outlier and overstates what a typical employee earns — in fact 8 of the 9 employees earn below the mean. The mode (₹5 lakhs, appearing twice) is too low and unstable to be a reliable summary. Because the median is resistant to extreme values, it gives the most honest picture of central salary in a right-skewed, outlier-affected dataset like this one.

Python Verification

PYTHON

```
import numpy as np
from scipy import stats

data = np.array([4, 5, 5, 6, 7, 8, 8, 9, 30])

mean = np.mean(data)
median = np.median(data)
mode = stats.mode(data, keepdims=True)
data_range = np.ptp(data)
variance = np.var(data)          # population variance
std_dev = np.std(data)           # population std dev

q1 = np.percentile(data, 25)
q3 = np.percentile(data, 75)
iqr = q3 - q1
lower_bound = q1 - 1.5 * iqr
upper_bound = q3 + 1.5 * iqr
outliers = data[(data < lower_bound) | (data > upper_bound)]

print("Mean:", round(mean, 2))
print("Median:", median)
print("Mode:", mode.mode[0], "(count:", mode.count[0], ")")
print("Range:", data_range)
print("Variance:", round(variance, 2))
print("Standard Deviation:", round(std_dev, 2))
print("Q1:", q1, " Q3:", q3, " IQR:", iqr)
print(f"Outlier bounds: ({lower_bound}, {upper_bound})")
print("Outliers:", outliers)
```

```
Mean: 9.11
Median: 7.0
Mode: 5 (count: 2 )
Range: 26
Variance: 56.99
Standard Deviation: 7.55
Q1: 5.0  Q3: 8.0  IQR: 3.0
Outlier bounds: (0.5, 12.5)
Outliers: [30]
```

Case Study 2 — Website Response Time

Data (ms, n = 10): 120, 125, 128, 130, 132, 135, 140, 145, 150, 300

Mean	Median	Range	Variance	Std. Dev.
150.5 ms	133.5 ms	180 ms	2558.05	50.58 ms

Skewness

The data is right-skewed (positively skewed). Nine of the ten requests cluster tightly between 120 and 150 ms, while one extreme value (300 ms) stretches the right tail. This pulls the mean (150.5 ms) well above the median (133.5 ms), the classic signature of right skew.

Why the Median May Be More Useful

The median is unaffected by the single slow request and reflects the response time a typical user actually experiences (≈134 ms). The mean is distorted by the 300 ms outlier and would lead an engineer to believe the "typical" response is far slower than it really is — a misleading basis for setting performance targets or SLAs.

Python Verification

PYTHON

```
import numpy as np

data = np.array([120, 125, 128, 130, 132, 135, 140, 145, 150, 300])

mean = np.mean(data)
median = np.median(data)
data_range = np.ptp(data)
variance = np.var(data)
std_dev = np.std(data)

print("Mean:", mean)
print("Median:", median)
print("Range:", data_range)
print("Variance:", round(variance, 2))
print("Standard Deviation:", round(std_dev, 2))
print("Skew direction:", "Right-skewed" if mean > median else "Left-skewed")
```

```
Mean: 150.5
Median: 133.5
Range: 180
Variance: 2558.05
Standard Deviation: 50.58
Skew direction: Right-skewed
```

Case Study 3 — Student Performance

Data (marks, n = 12): 42, 45, 48, 50, 52, 52, 55, 58, 60, 62, 65, 95

Q1	Q2 (Median)	Q3	IQR	Mean
49.5	53.5	60.5	11	57.0

Outlier Check (1.5 × IQR Rule)

Lower bound = Q1 − 1.5(IQR) = 49.5 − 16.5 = 33.0. Upper bound = Q3 + 1.5(IQR) = 60.5 + 16.5 = 77.0. The score of 95 lies above the upper bound and is flagged as an outlier; no value lies below the lower bound.

Effect of the Outlier on the Mean

The mean (57.0) sits noticeably above the median (53.5) because the outlier score of 95 pulls the arithmetic average upward, even though it represents only one of twelve students. Since the mean uses every value with equal weight, a single extreme score has an outsized effect on it, while the median — based only on rank position — stays close to where most students actually scored.

Python Verification

PYTHON

```
import numpy as np

data = np.array([42, 45, 48, 50, 52, 52, 55, 58, 60, 62, 65, 95])

q1 = np.percentile(data, 25)
q2 = np.percentile(data, 50)
q3 = np.percentile(data, 75)
```

```
iqr = q3 - q1
lower_bound = q1 - 1.5 * iqr
upper_bound = q3 + 1.5 * iqr
outliers = data[(data < lower_bound) | (data > upper_bound)]

mean = np.mean(data)
median = np.median(data)

print("Q1:", q1)
print("Q2 (Median):", q2)
print("Q3:", q3)
print("IQR:", iqr)
print(f"Outlier bounds: ({lower_bound}, {upper_bound})")
print("Outliers:", outliers)
print("Mean:", round(mean, 2))
print("Median:", median)
```

```
Q1: 49.5
Q2 (Median): 53.5
Q3: 60.5
IQR: 11.0
Outlier bounds: (33.0, 77.0)
Outliers: [95]
Mean: 57.0
Median: 53.5
```

Case Study 4 — E-Commerce Orders

Data (orders/day, n = 10): 18, 20, 22, 22, 24, 25, 26, 28, 30, 35

Mean	Median	Mode	Variance	Std. Dev.	CV
25.0	24.5	22	22.80	4.78	19.1%

What the Standard Deviation Tells the Company

A standard deviation of about 4.78 orders means daily order counts typically fall within roughly ±5 orders of the 25-order average — i.e. most days see between about 20 and 30 orders. It quantifies day-to-day unpredictability in demand, which matters for staffing and inventory planning.

Variability Assessment

The coefficient of variation, CV = (σ / μ) × 100 = (4.78 / 25.0) × 100 ≈ 19.1%, falls in the moderate range (commonly, CV under ~15% is considered low variability and above ~30% is considered high). At 19.1%, the order data is best described as moderately variable — demand fluctuates from day to day but without wild, unpredictable swings.

Python Verification

PYTHON

```
import numpy as np

data = np.array([18, 20, 22, 22, 24, 25, 26, 28, 30, 35])

from scipy import stats
mean = np.mean(data)
median = np.median(data)
mode = stats.mode(data, keepdims=True)
variance = np.var(data)
std_dev = np.std(data)
cv = (std_dev / mean) * 100

print("Mean:", mean)
print("Median:", median)
print("Mode:", mode.mode[0])
print("Variance:", round(variance, 2))
print("Standard Deviation:", round(std_dev, 2))
print("Coefficient of Variation (%):", round(cv, 2))
```

```
Mean: 25.0
Median: 24.5
Mode: 22
Variance: 22.8
Standard Deviation: 4.77
Coefficient of Variation (%): 19.1
```

Case Study 5 — Comparing Two Data Science Models

Model A errors (n = 8): 2, 3, 3, 4, 4, 5, 5, 6

Model B errors (n = 8): 1, 1, 2, 4, 5, 7, 8, 9

Model	Mean Error	Variance	Std. Dev.
Model A	4.000	1.500	1.225
Model B	4.625	8.734	2.955

Consistency

Model A has both the lower variance (1.50 vs 8.73) and lower standard deviation (1.23 vs 2.96), so its prediction errors are tightly clustered around its mean. Model A is the more consistent, more predictable model.

Does a Lower Mean Error Mean "Better"?

Not necessarily. Model A also happens to have the lower mean error here, but mean error and consistency are two different properties: a model could have a low mean error purely by having a few very small errors offset by a few large ones, while a slightly higher-mean model with low variance behaves reliably every time. In applications where predictable, stable performance matters (e.g. safety-critical systems), a model with marginally higher mean error but much lower variance can be the better practical choice.

Python Verification

PYTHON

```
import numpy as np

model_a = np.array([2, 3, 3, 4, 4, 5, 5, 6])
model_b = np.array([1, 1, 2, 4, 5, 7, 8, 9])

for name, data in [("Model A", model_a), ("Model B", model_b)]:
    mean = np.mean(data)
    variance = np.var(data)
    std_dev = np.std(data)
    print(f"{name} -> Mean: {mean:.3f}, Variance: {variance:.3f}, Std Dev: {std_dev:.3f}")
```

```
Model A -> Mean: 4.000, Variance: 1.500, Std Dev: 1.225
Model B -> Mean: 4.625, Variance: 8.734, Std Dev: 2.955
```